

Project Reflections Document

Ossi Korpela

K438761

Ossi.korpela@tuni.fi

Instructions for the teaching assistant:

1. Implemented optional features

Some tests for the services

2. Instructions for examiner to test the system.

Should run as specified in the instructions (care pdf formatting issues):

"docker-compose build --no-cache"

"docker-compose up -d"

If you want to run the tests:

```
docker build -t python-pytest tests
docker run --rm python-pytest
```

Pipeline repo: git@compse140.devops-gitlab.rd.tuni.fi:ossikorpela/gl.git

Data about the platform you used in development (hardware, CPU architecture, operating system, version of docker and docker-compose)

AMD Ryzen 5 5600X

AMD64

Windows 11

```
Docker Compose version v2.30.3-desktop.1
Docker version 27.3.1, build ce12230
```

Description of the CI/CD pipeline

1. First the pipeline tries to build the docker images.
2. Then the pipeline runs the tests

The tests are run with docker (not compose) separately from the application.

3. The pipeline deploys the system

Briefly document all steps:

- Version management; use of branches etc

The final project is in the branch “project”

- Building tools

Docker and docker-compose

- Testing; tools and test cases

Pytest was used for testing with the pytest-order plugin.

- Deployment

Deployment is simply done with docker-compose.

Example runs of the pipeline

 Failed 00:01:39 1 day ago	add tests for run log #3460 project 51288a69		
 Passed 00:02:35 1 day ago	fix ex4 functionality #3456 project 39de8d01		
 Failed 00:01:41 1 day ago	add test for basic fetch #3454 project 16372832		
 Passed 00:02:45 2 days ago	pipeline test #3366 project 43cd88f8		

Include some kind of log of both failing test and passing.

```

144 #11 unpacking to docker.io/library/python-pytest:latest 0.3s done
145 #11 DONE 0.5s
146 $ docker run --rm python-pytest
147 ===== test session starts =====
148 platform linux -- Python 3.10.16, pytest-8.3.4, pluggy-1.5.0
149 rootdir: /app
150 plugins: order-1.3.0
151 collected 8 items
152 api_tests.py ..... [100%]
153 ===== 8 passed in 10.45s =====
154 $ docker-compose down
155 Container gl-nginx-1 Stopping
156 Container gl-service2-1 Stopping
157 Container gl-nginx-1 Stopped
158 Container gl-nginx-1 Removing
159 Container gl-nginx-1 Removed
160 Container gl-service1_3-1 Stopping
161 Container gl-service1_1-1 Stopping
162 Container gl-service1_2-1 Stopping
163 Container gl-service1_1-1 Stopped
164 Container gl-service1_1-1 Removing
165 Container gl-service1_1-1 Removed
166 Container gl-service1_2-1 Stopped
167 Container gl-service1_2-1 Removing
168 Container gl-service1_2-1 Removed
169 Container gl-service1_3-1 Stopped
170 Container gl-service1_3-1 Removing
171 Container gl-service1_3-1 Removed
172 Container gl-service2-1 Stopped
173 Container gl-service2-1 Removing
174 Container gl-service2-1 Removed
175 Network gl_ex4-net Removing
176 Network gl_ex4-net Removed
177 Cleaning up project directory and file based variables
178 Job succeeded

```

```

195 > assert response.status_code == 200, "Expected 200 from /run-log endpoint"
196 E   AssertionError: Expected 200 from /run-log endpoint
197 E   assert 500 == 200
198 E   + where 500 = <Response [500]>.status_code
199 api_tests.py:85: AssertionError
200 ----- test_run_log_changes -----
201 @pytest.mark.order(6)
202 def test_run_log_changes():
203     """
204     Should return the run log containing correct state changes
205     """
206     response = put_to("INIT")
207 > assert response.status_code == 200, "state changes should work as before"
208 E   AssertionError: state changes should work as before
209 E   assert 500 == 200
210 E   + where 500 = <Response [500]>.status_code
211 api_tests.py:94: AssertionError
212 ----- test_paused_state_changes -----
213 @pytest.mark.order(7)
214 def test_paused_state_changes():
215     response = put_to("INIT")
216 > assert response.status_code == 200, "state changes should work as before"
217 E   AssertionError: state changes should work as before
218 E   assert 500 == 200
219 E   + where 500 = <Response [500]>.status_code
220 api_tests.py:107: AssertionError
221 ===== short test summary info =====
222 FAILED api_tests.py::test_initial_state - AssertionError: assert '<ldoctype h...
223 FAILED api_tests.py::test_state_change_admin - AssertionError: Expected 200 w...
224 FAILED api_tests.py::test_ex4_functionality - AssertionError: state changes s...
225 FAILED api_tests.py::test_run_log - AssertionError: Expected 200 from /run-lo...
226 FAILED api_tests.py::test_run_log_changes - AssertionError: state changes sho...
227 FAILED api_tests.py::test_paused_state_changes - AssertionError: state change...
228 ===== 6 failed, 1 passed in 0.28s =====
229 Cleaning up project directory and file based variables
230 ERROR: Job failed: exit code 1

```

Reflections

It took quite a lot of time for me to get the runner + docker setup going on windows, and even after getting it to work, although having learnt more about runners, I was a little unsatisfied as working around a strange bug doesn't really teach you anything other than more googling. This was just my own stubbornness/stupidity for not using a linux vm

I learnt a lot about orchestrating more complicated applications, and for example in the process of trying to get the shutdown and state features to work I learnt a lot about the

ways that containers interact. I think these things were the most difficult ones to get working (I tried a lot of things that didn't work until I got these working).

Also I learnt about testing more complex applications. Most of the tests that I had written before this were kind of static. Here there were multiple services which might even be busy as they are bombarded with multiple tests. The pytest library wasn't really well-suited for this, even with the pytest-order plugin. I had to put everything in a single file and use sleep calls for the delays. I would look for another library if I were to test something similar again.

And of course, I learnt about pipelines. Building one gave a pretty good idea of how they work and how one would go about creating or expanding one.

If I were to change something it would be how the state is handled. Right now it's just a lazy volume+file shared between the service1s. As it stands it's not robust for simultaneous calls. I would consider using locks for a start or a db in case of a bigger system.

All in all, this was the most difficult project I had done during my studies. As a result, I probably also learnt the most :)

Amount effort (hours) used :

estimate: 50 hours